ICA Bienestar

Entrega del proyecto · Semana 12

Bienestar animal, evaluado en campo.

Código del proyecto, repositorio Git, diseños de interfaz, diagramas UML y diseño de la base de datos.

Asignatura	Aplicación a la Inteligencia Artificial
Docente	PhD. Paola Andrea Valencia A.
Responsable	Juan Esteban Niño Naranjo · arquitectura y módulo de seguridad
Repositorio	github.com/98765432100123456/Aplicacion-a-la-inteligencia-artificial-
Versión	v0.2.0 · octubre de 2026

Resumen de la entrega

ICA Bienestar Animal es una aplicación web para evaluar el bienestar animal en fincas con listas de chequeo por especie, puntaje automático, diagnóstico y recomendaciones con inteligencia artificial. Esta entrega cierra el primer sprint: el módulo de seguridad funciona de extremo a extremo y el resto del sistema queda diseñado.

70	29	10	32
pruebas automatizadas en verde	tablas en el modelo de datos	diagramas UML	capturas de interfaz

Evidencias solicitadas

Evidencia	Dónde se encuentra
Código del proyecto	`backend/` (NestJS 11, TypeORM, PostgreSQL) y `frontend/` (Angular 20). Organización descrita en la sección Código del proyecto.
Repositorio Git	Rama `main`, historial de commits descriptivos, integración continua en GitHub Actions y etiqueta de versión `v0.2.0`.
Diseños de interfaz	`docs/disenos/`: ocho pantallas en escritorio y teléfono, modo claro y oscuro. Sistema visual en `docs/SISTEMA-DE-DISENO.md`.
UML	`docs/uml/DIAGRAMAS-UML.md`: casos de uso, clases, secuencia, actividad, estados, componentes y despliegue.
Diseño de la base de datos	`docs/base-de-datos/`: diagramas entidad relación, script SQL validado y diccionario de datos.

Estado por módulo

Módulo	Estado	Requerimientos
Seguridad	● Implementado	RF01 a RF04, RF34 parcial, RNF05, RNF07, RNF08, RNF12 a RNF14
Fincas y catálogos	● Diseñado	RF05 a RF08, RF35, RF38
Parametrización de especies	● Diseñado	RF09 a RF12, RNF17, RNF28
Evaluación	● Diseñado	RF13 a RF21, RF36
Diagnóstico y recomendaciones con IA	● Diseñado	RF22 a RF27, RNF25, RNF26
Reportes y certificación	● Diseñado	RF28 a RF33, RF38, RF40

Código del proyecto

Dos aplicaciones independientes que se comunican por HTTPS con una API REST. Cada carpeta tiene una sola responsabilidad y cada módulo del servicio sigue el patrón controlador, servicio y entidad.

```
backend/src
├── auth/          ingreso, token, recuperación
├── users/         gestión de usuarios
├── roles/         roles y catálogo de permisos
├── audit/         bitácora de auditoría
├── mail/          envío de correos
├── common/        guardas, decoradores,
│                  política de contraseñas
├── config/        variables de entorno validadas
├── database/      migraciones y datos iniciales
└── health/        verificación de estado
```

```
frontend/src/app
├── core/          sesión, API, interceptores,
│                  guardas de rutas, modelos
├── shared/        logo, diálogo, mensajes,
│                  encabezado, contraseña
├── layout/        barra superior y menú
└── features/
    ├── auth/      ingreso y recuperación
    ├── home/      inicio
    ├── users/     usuarios
    ├── roles/     roles y permisos
    └── account/   cuenta
```

Decisiones de organización

Práctica	Aplicación en el código
Módulos por funcionalidad	Cada requerimiento vive en su carpeta; los módulos futuros (fincas, evaluación) se agregan sin tocar los existentes.
Seguridad por defecto	Tres guardas globales: límite de peticiones, token JWT y permisos. Una ruta es pública solo si se marca con `@Public()`.
Validación en ambos extremos	DTO con `class-validator` en la API y formularios reactivos en Angular con los mismos mensajes.
Configuración externa	Variables de entorno validadas al arrancar; en producción las credenciales vienen de Secret Manager.
Esquema versionado	La base solo cambia mediante migraciones de TypeORM.
Componentes reutilizables	Diálogo, mensajes y encabezado compartidos; el sistema visual se define una vez en `styles.scss`.
Modo demostración	Un interceptor simula la API en el navegador para revisar la interfaz sin servidor ni base de datos.

Pruebas

Conjunto	Pruebas	Qué verifica
Unitarias del servicio (Jest)	3	Guarda de permisos
Integración del servicio (Jest y Supertest)	21	Ingreso, bloqueo, recuperación, usuarios, roles y permisos contra PostgreSQL real
Unitarias de la interfaz (Karma)	9	Sesión, política de contraseñas y mensajes de error
Extremo a extremo (Playwright y axe-core)	37	Flujos completos en escritorio y teléfono, accesibilidad WCAG 2.1 AA, modo oscuro, texto al 200 % y teclado

Repositorio Git

Todo el proyecto está en la rama `main`: código, documentación, diagramas y capturas. Cada commit describe un cambio completo y verificable, y la integración continua compila y prueba cada envío.

Historial de commits

Commit	Fecha	Descripción
`9e7890b`	2026-10-17	Diagramas UML y diseño completo de la base de datos
`f54ad06`	2026-10-17	Rediseño visual de la interfaz con identidad propia
`2580160`	2026-10-15	Aclara la descripción del comando para teléfono móvil
`bde26aa`	2026-10-15	Modo demostración: ver la aplicación sin backend ni base de datos
`7f1e714`	2026-10-15	Manual de instalación: descarga desde la rama main
`95c6330`	2026-10-15	Documentación formal sin numerales
`ae142c0`	2026-10-14	Aplica las guías de Apple (HIG) al frontend e instala skills de diseño
`2fa2c26`	2026-10-14	Frontend Angular, infraestructura y documentación de arquitectura
`054383d`	2026-10-14	Backend NestJS: autenticación, roles, usuarios y recuperación de contraseña
`df4bc5f`	2026-10-09	Initial commit

Integración continua

El archivo `.github/workflows/ci.yml` ejecuta dos trabajos en cada envío a `main` y en cada solicitud de integración:

Trabajo	Pasos
Servicio de aplicación	Levanta PostgreSQL 16, instala dependencias, verifica tipos, ejecuta pruebas unitarias y de integración, y compila.
Interfaz web	Instala dependencias, ejecuta las pruebas unitarias en Chrome sin ventana y compila la versión de producción.

Buenas prácticas aplicadas

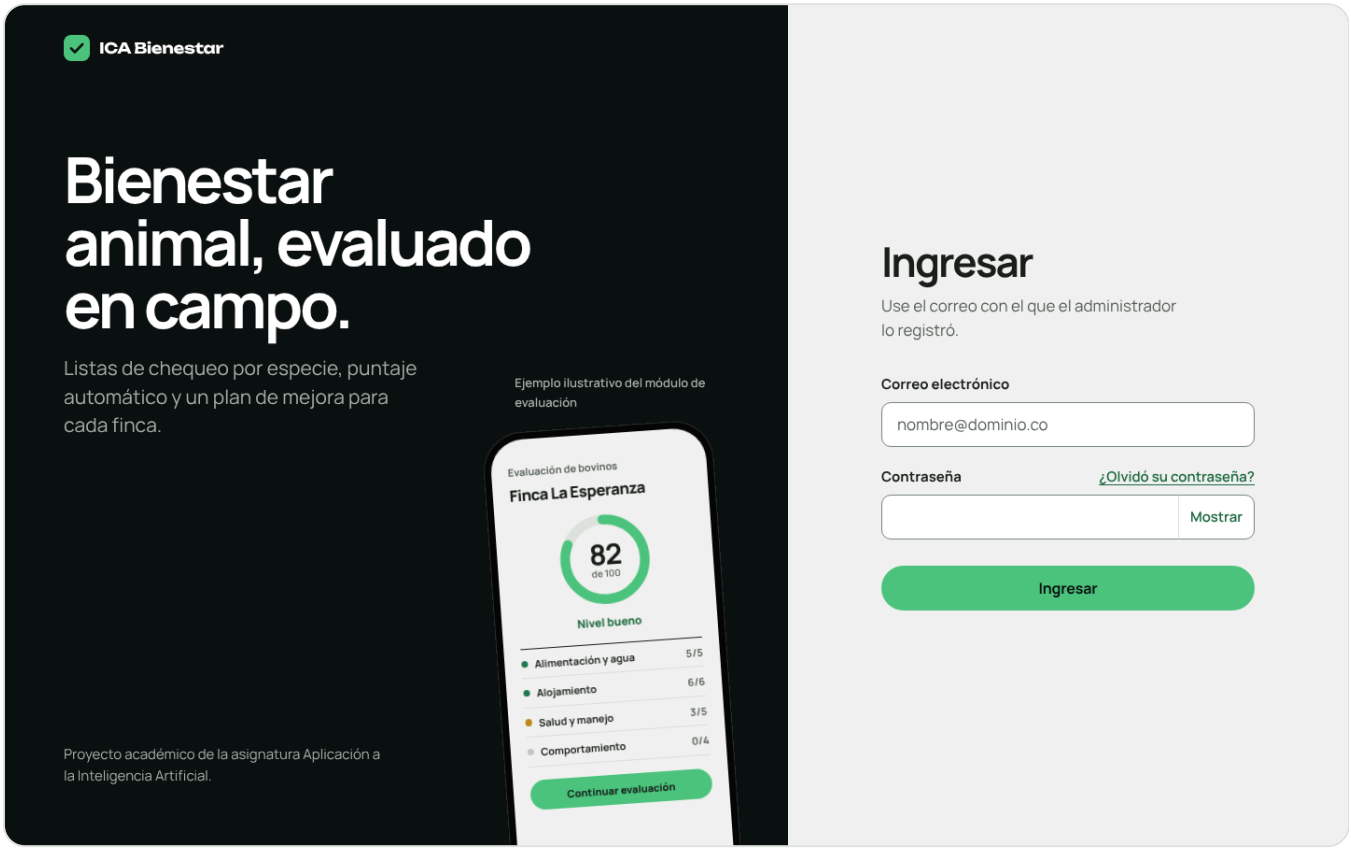
Práctica	Aplicación
Archivos excluidos	`.gitignore` excluye dependencias, compilados y archivos `.env`; el repositorio incluye `.env.example` sin secretos.
Versiones	Etiqueta `v0.2.0` sobre el commit de esta entrega.
Estrategia de ramas	Documentada en `docs/RAMAS.md` para los próximos sprints: `feature/<módulo>` desde `develop` y publicación en `main`.
Documentación junto al código	Los diagramas se guardan como código Mermaid y se versionan con el proyecto.

Diseños de interfaz

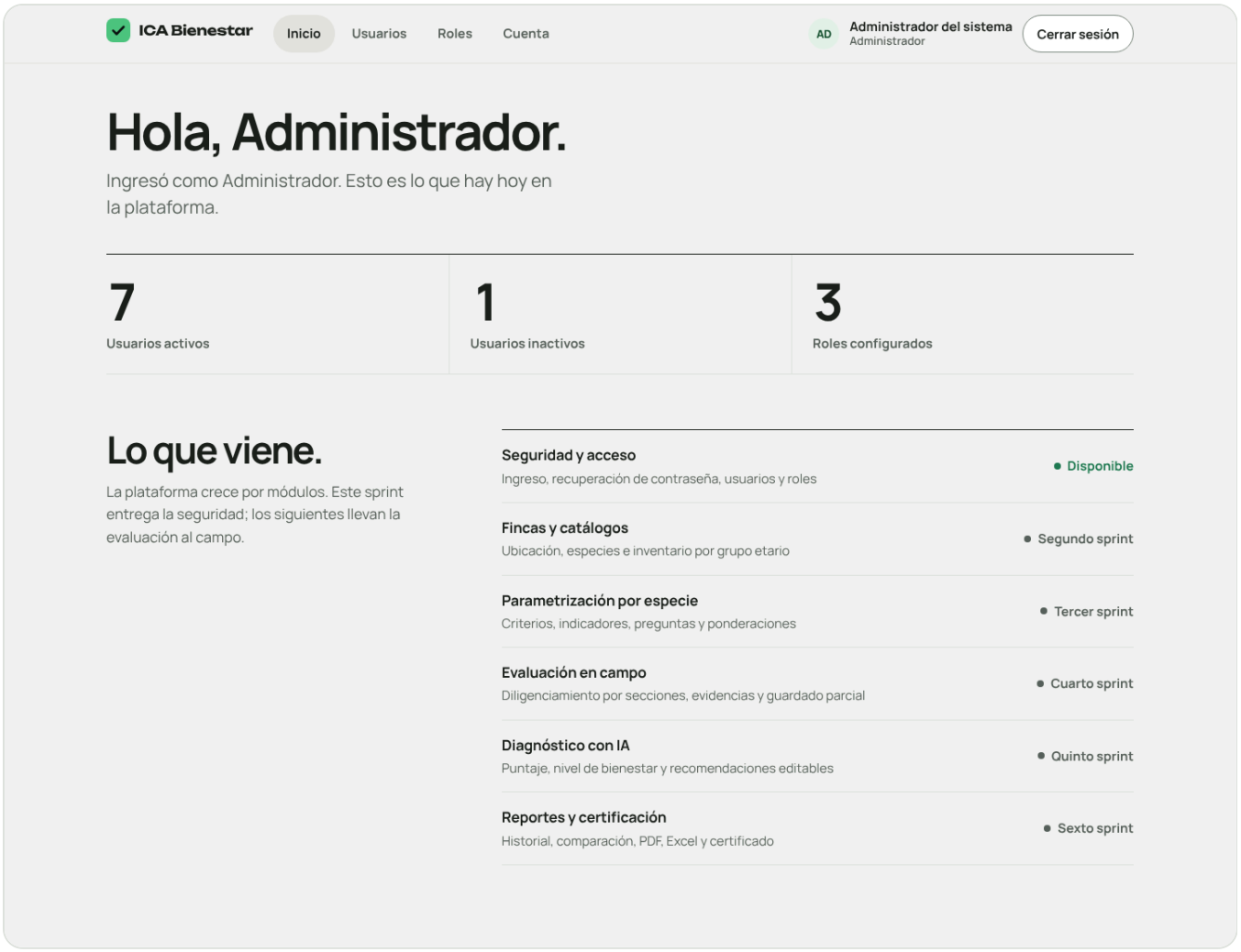
La interfaz muestra el producto antes de prometerlo y organiza la información como un libro de registro: titulares grandes, filas separadas por líneas y un solo verde para la acción principal. Se adapta a teléfono, tableta y computador, y sigue la preferencia de modo oscuro del dispositivo.

Fundamentos visuales

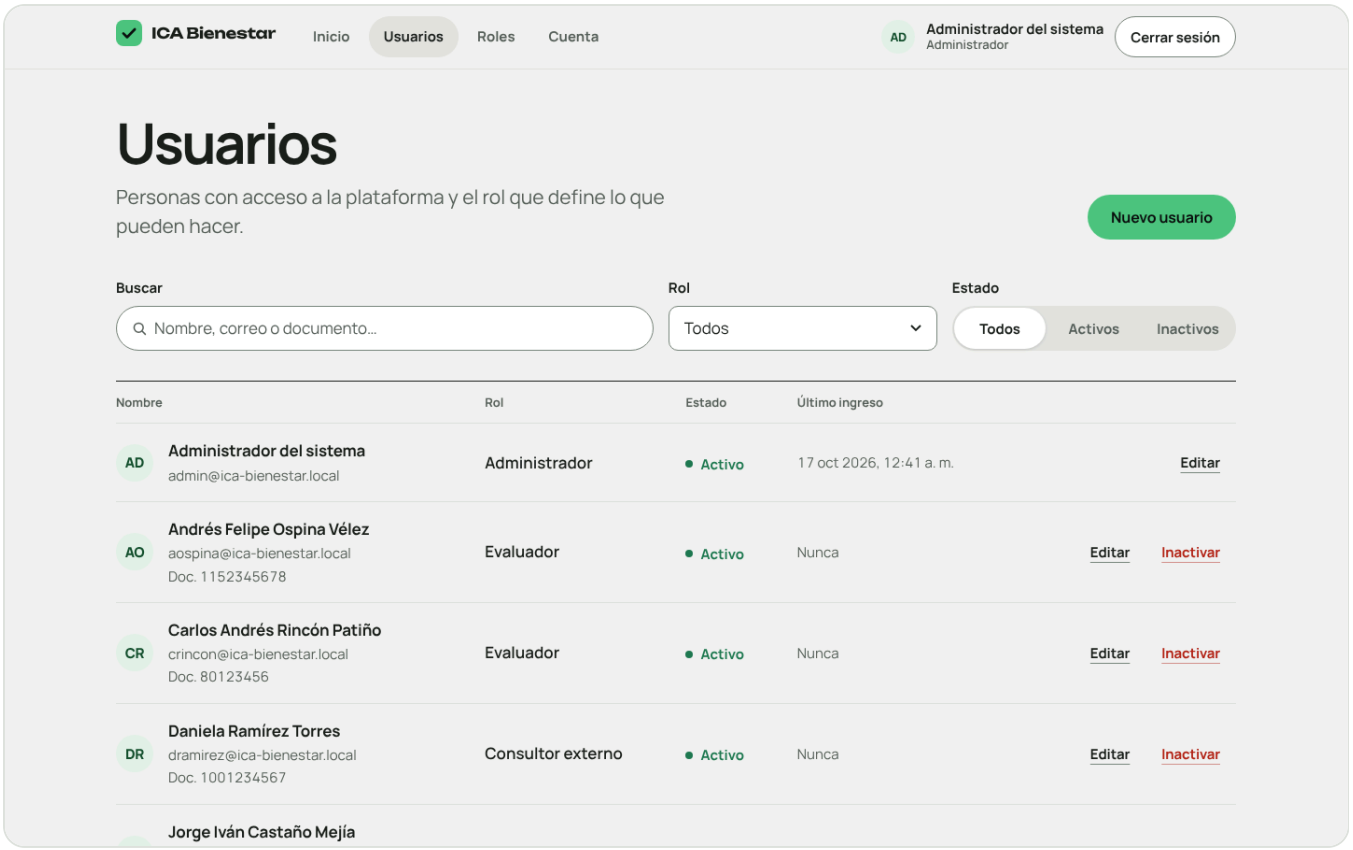
Elemento	Decisión
Tipografía	Manrope para la interfaz, con titulares de peso 800 y letras juntas; Unbounded solo en el logotipo.
Color	Verde pasto `#4cc77f` con texto oscuro (contraste 8.8:1) para la acción principal; fondo `#f2f3f0` y noche `#0e1210`.
Componentes	Botones en píldora, interruptores para permisos, control segmentado, punto de estado, hojas inferiores en el teléfono y barra superior translúcida.
Movimiento	Curva de salida `cubic-bezier(0.23, 1, 0.32, 1)`, presión a `scale(0.97)` y entradas escalonadas; se desactiva con movimiento reducido.
Accesibilidad	Cero violaciones WCAG 2.1 AA en axe-core, objetivos táctiles de 44 px, foco visible, texto ampliado al 200 % sin desbordamiento.



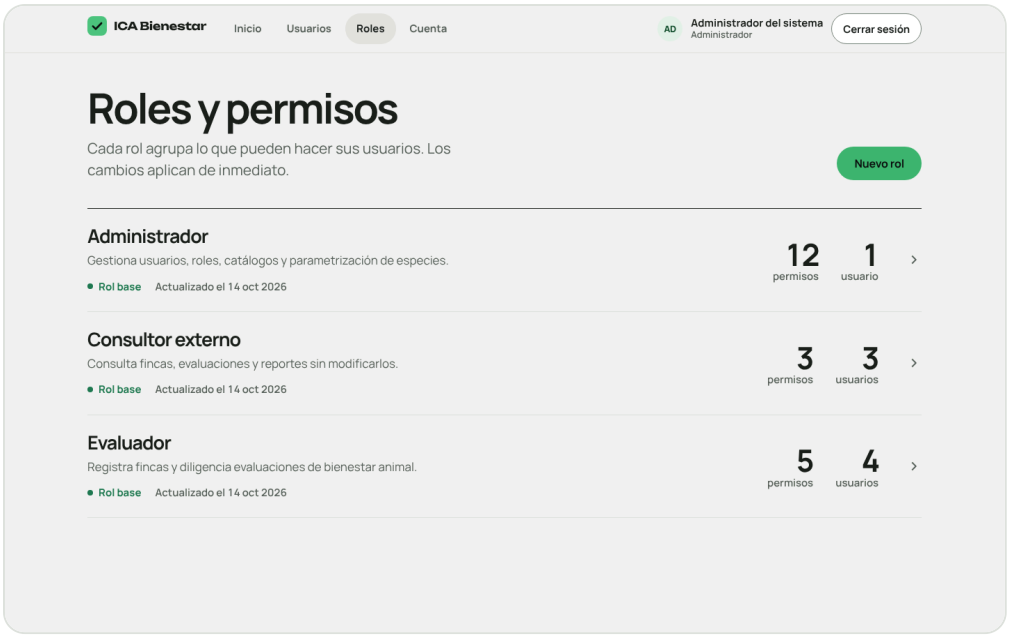
Ingreso en escritorio. El teléfono muestra un ejemplo ilustrativo del módulo de evaluación.



Inicio: cifras del sistema y hoja de ruta de los próximos sprints.



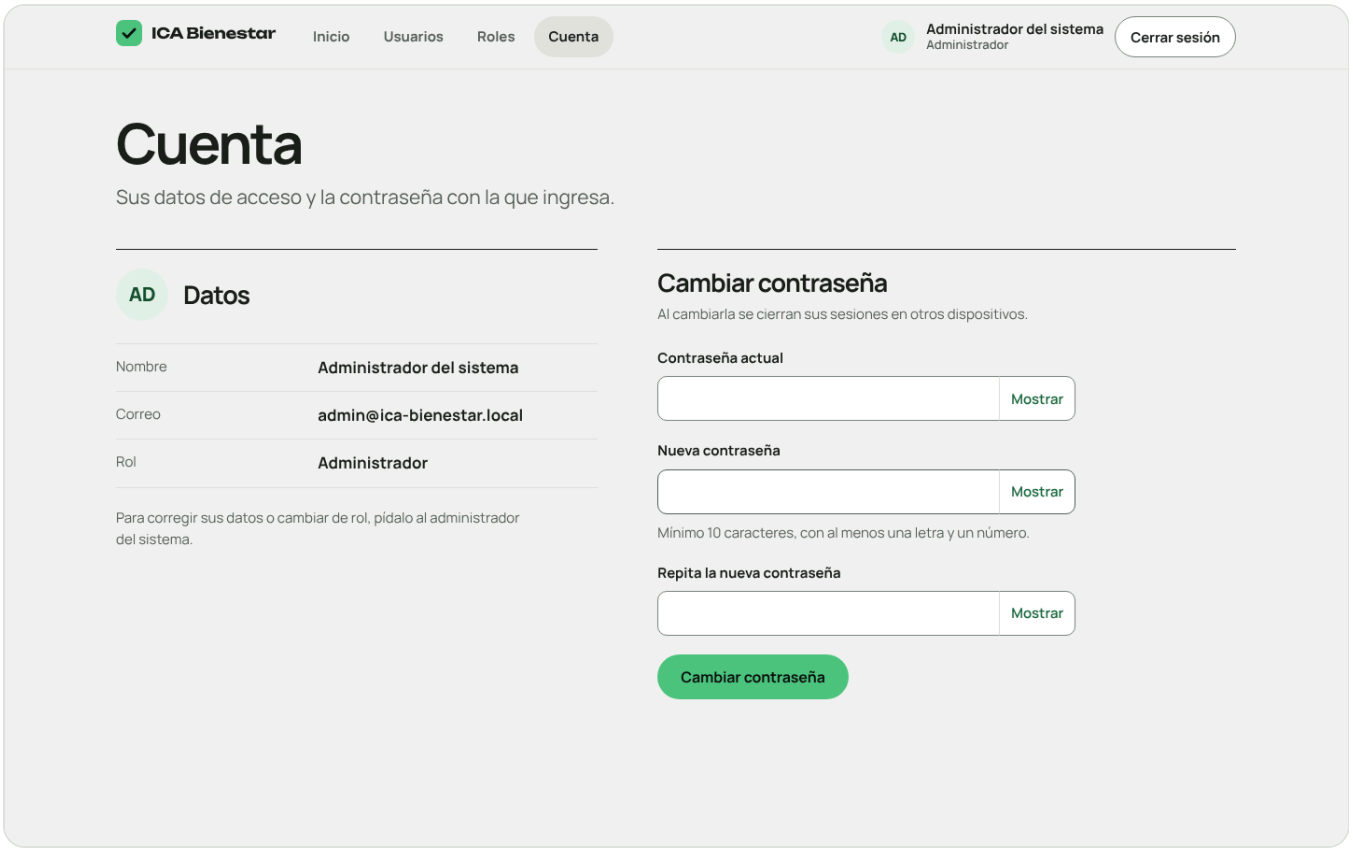
Gestión de usuarios con búsqueda, filtro por rol y control segmentado de estado.



Roles y permisos.

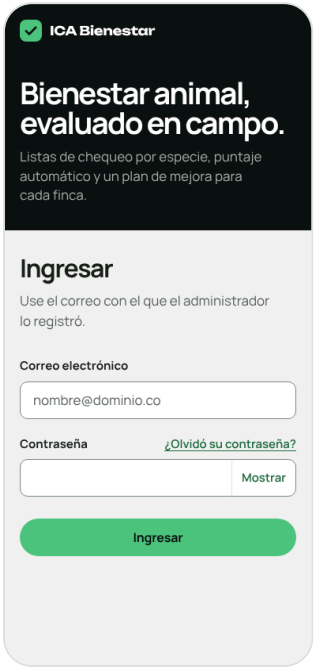


Editor de rol en el teléfono, con interruptores por módulo.



Cuenta: datos de la persona y cambio de contraseña.

Teléfono



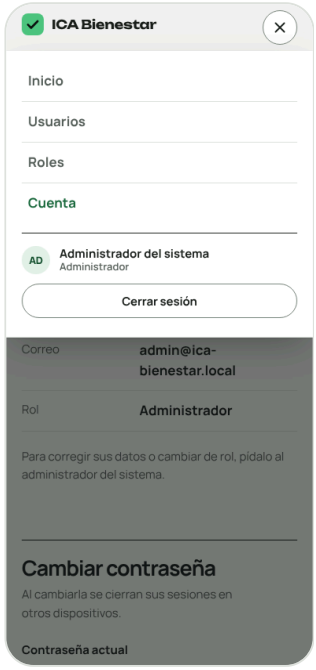
Ingreso



Usuarios como fichas

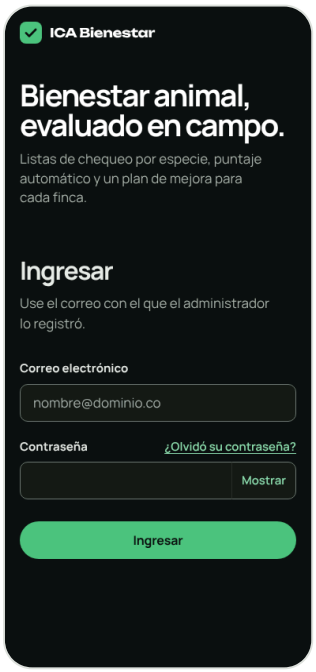


Formulario en hoja inferior

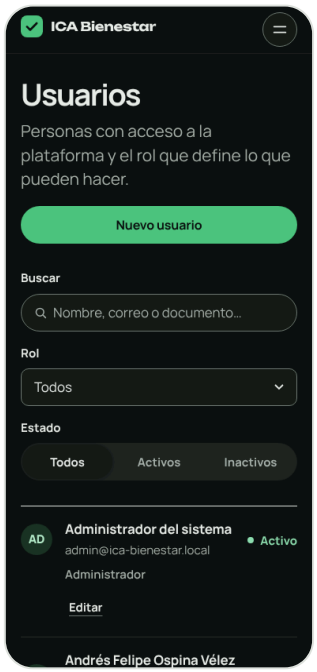


Menú desplegable

Modo oscuro



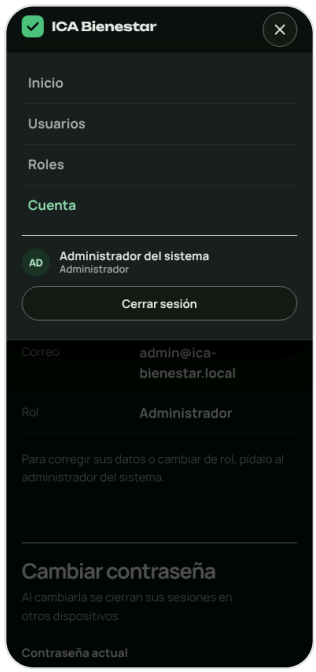
Ingreso



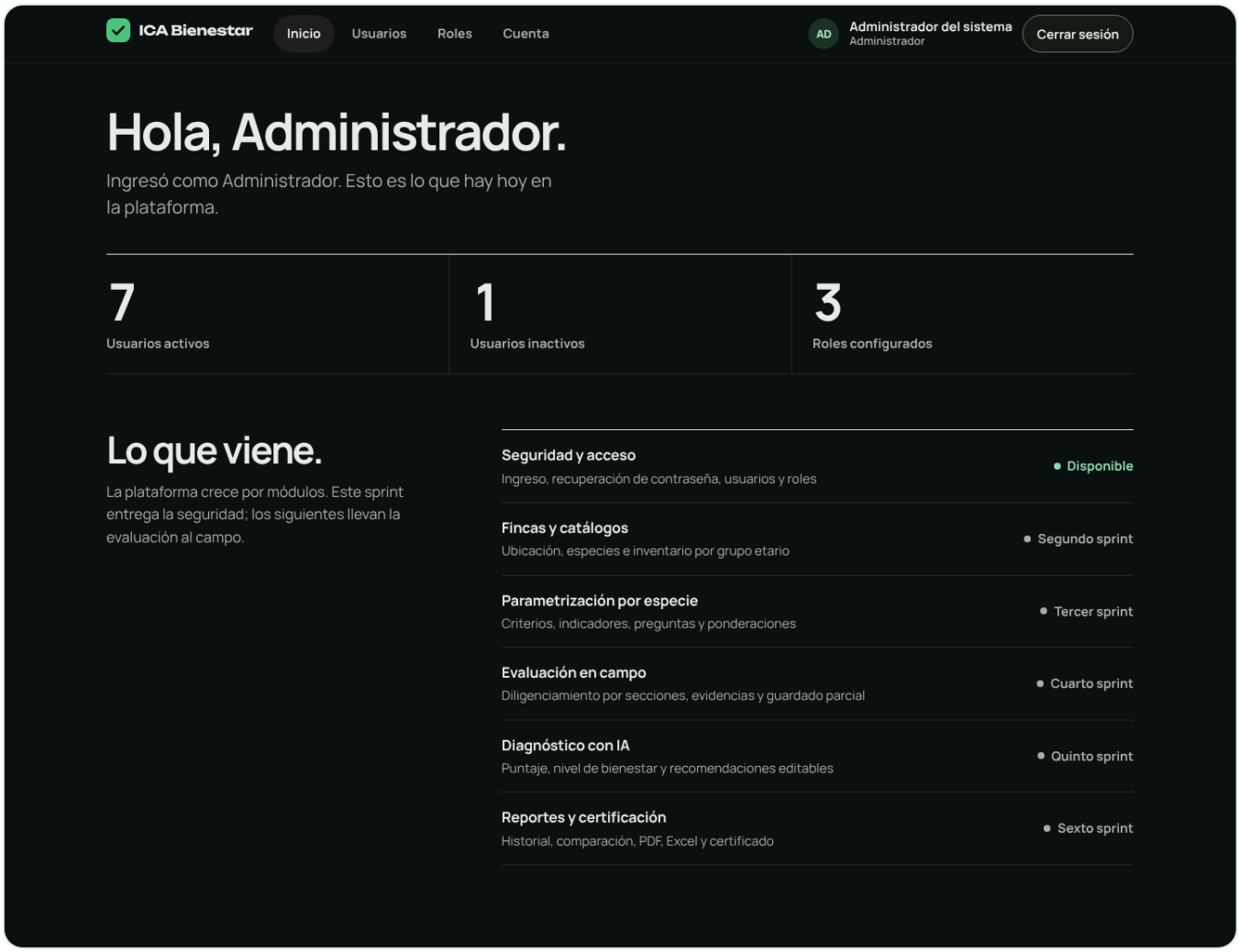
Usuarios



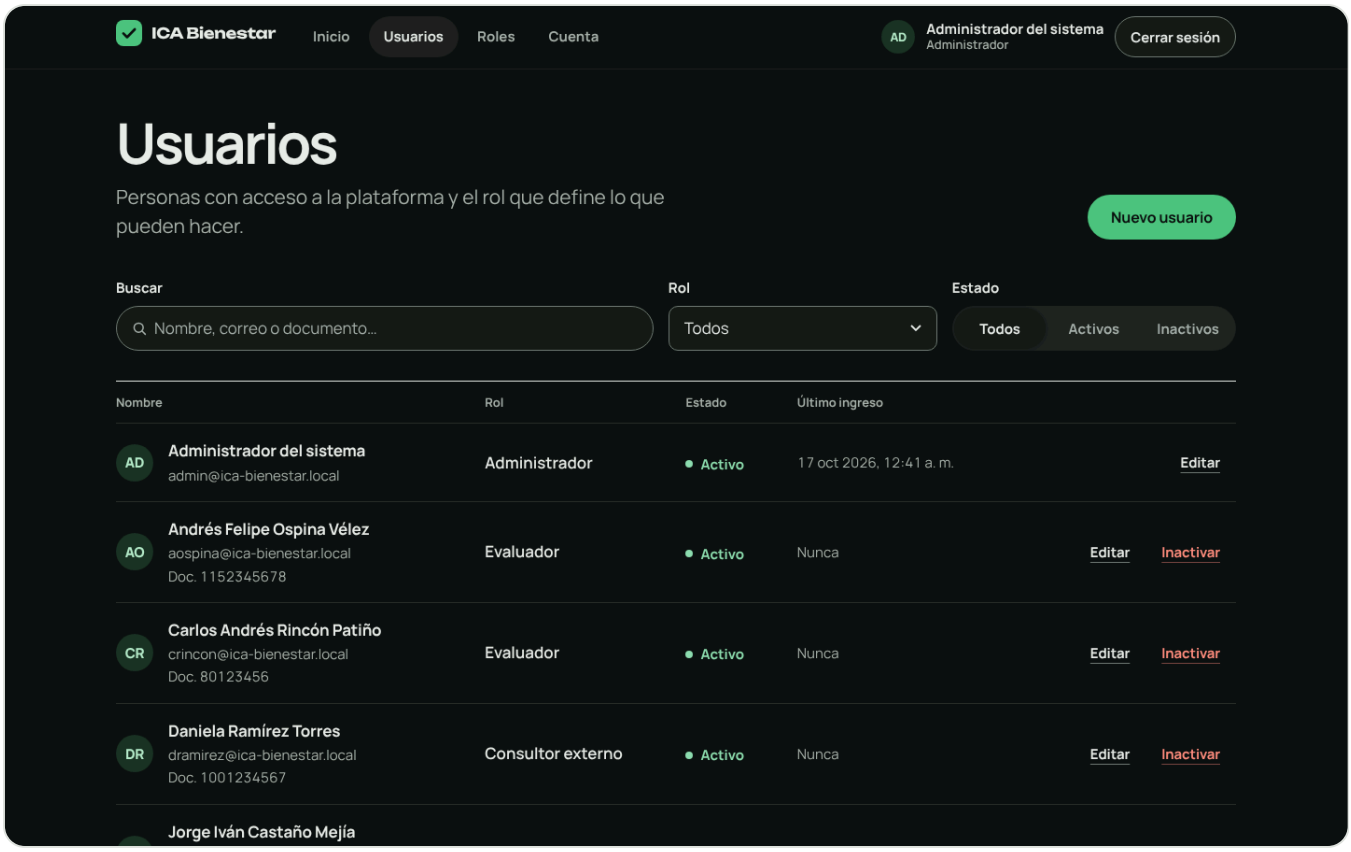
Formulario



Menú



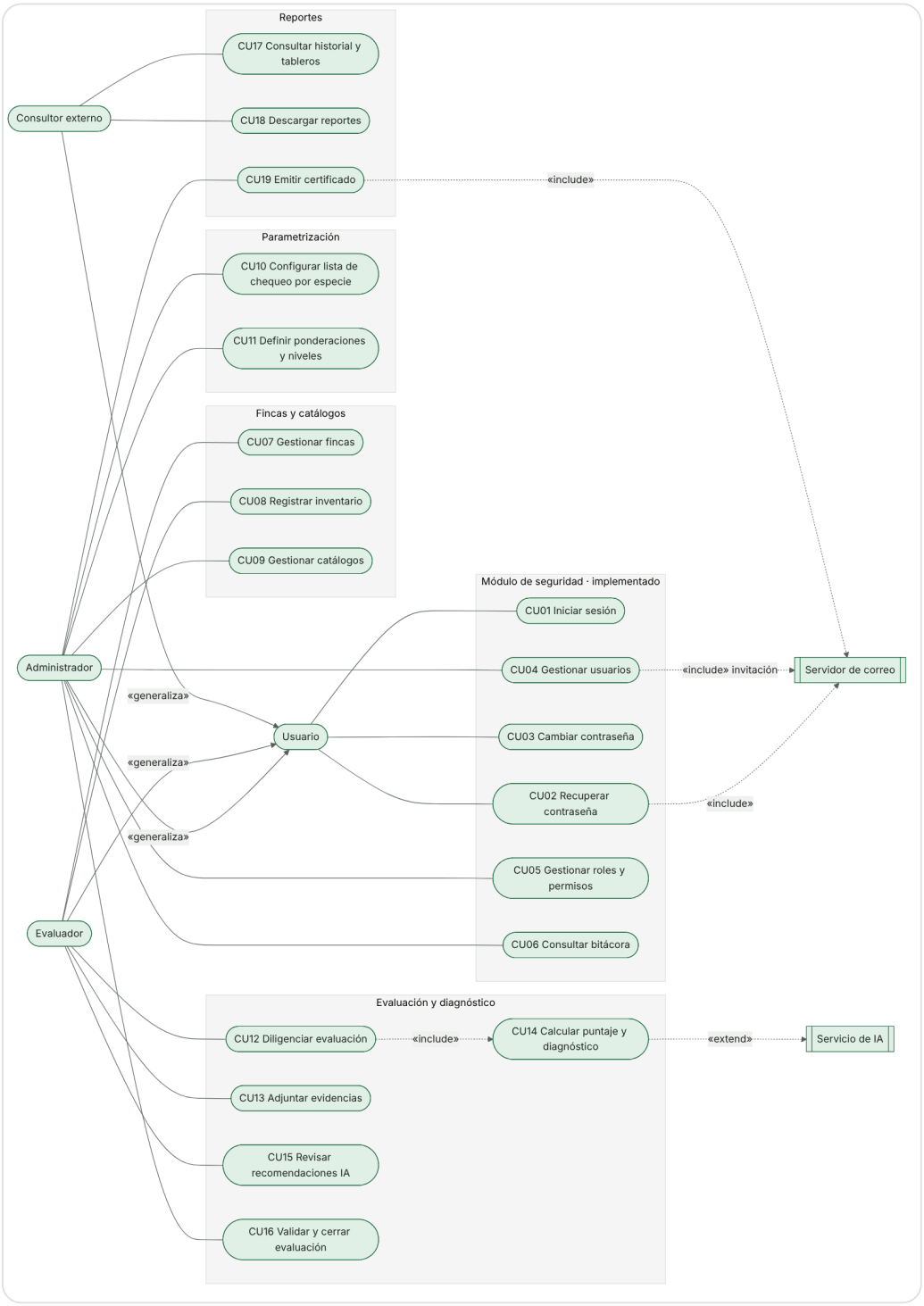
Inicio en modo oscuro.



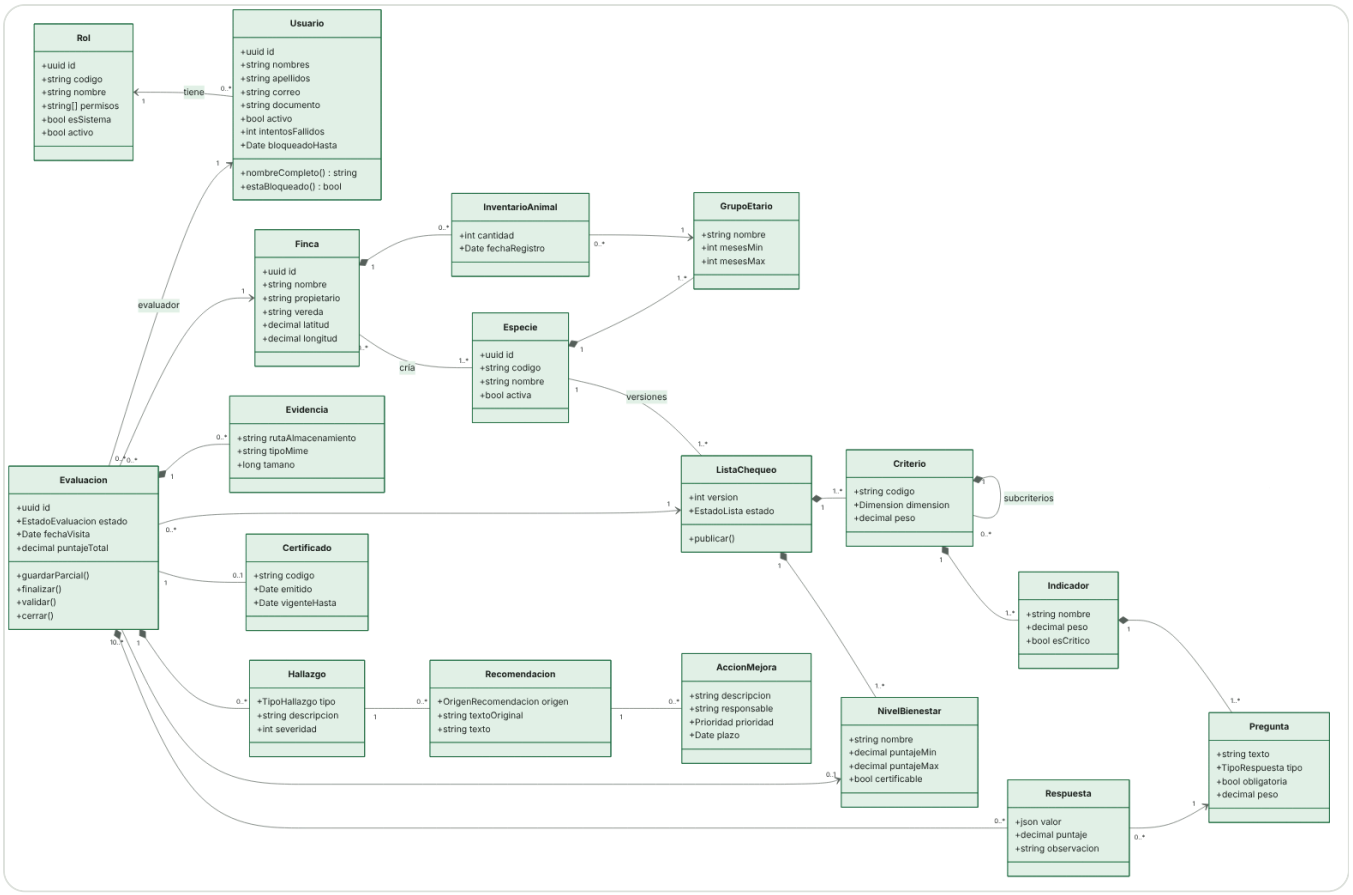
Usuarios en modo oscuro.

Diagramas UML

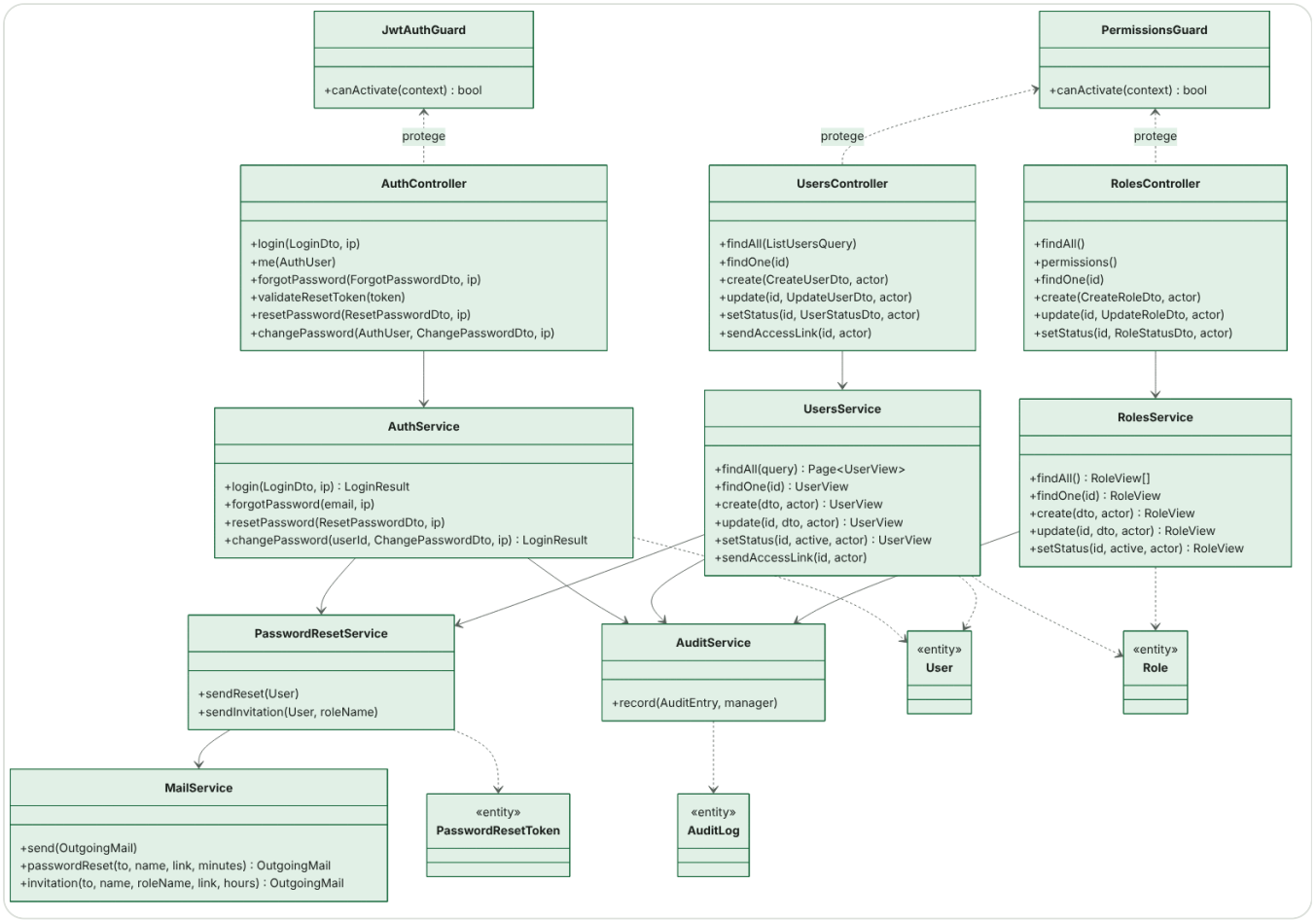
Los diagramas del módulo de seguridad describen el código existente; los de evaluación y diagnóstico describen el diseño de los próximos sprints. Todos se generan desde código Mermaid guardado en `docs/uml/fuentes/`.



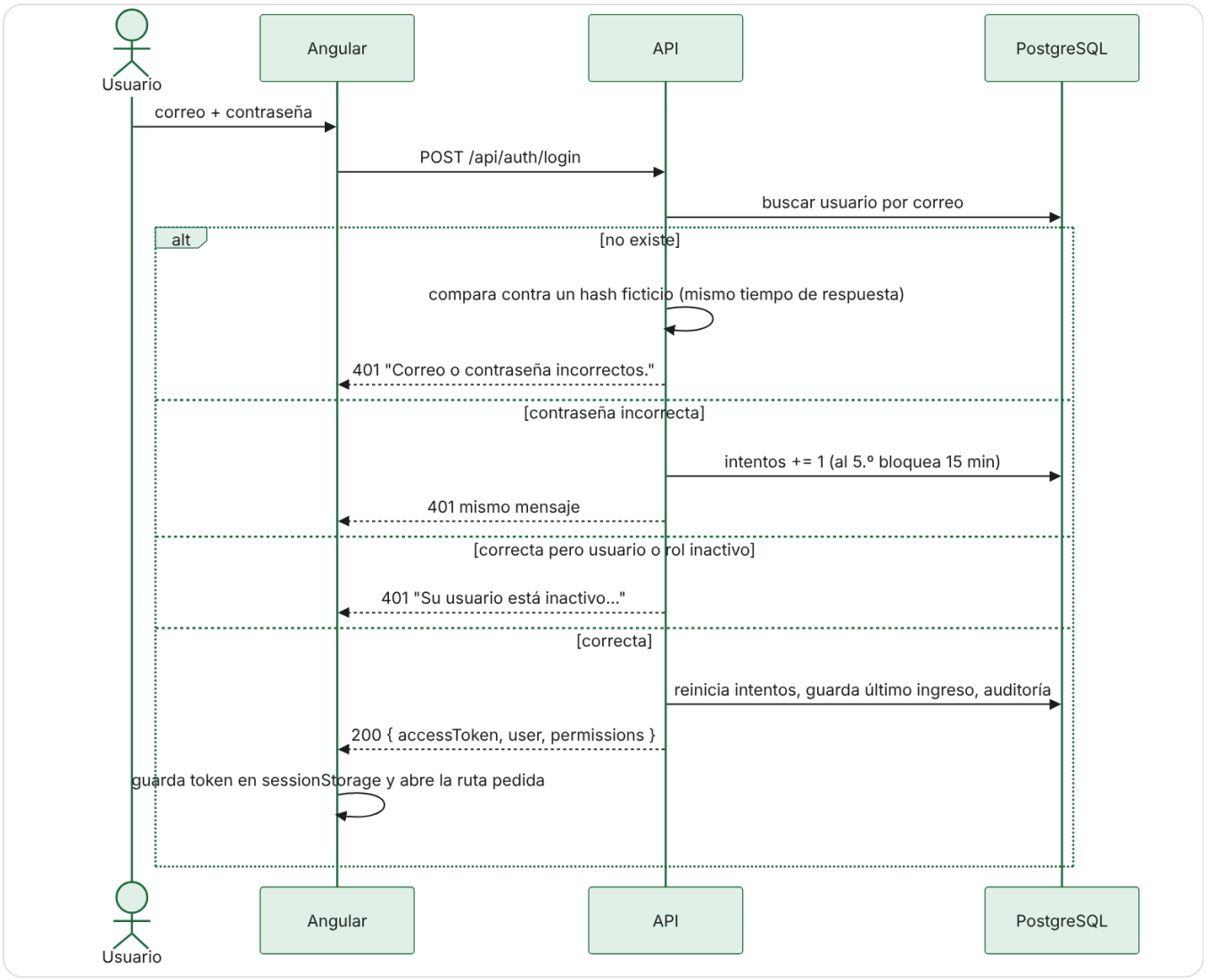
Casos de uso. Los tres actores heredan del actor Usuario el acceso a la plataforma.



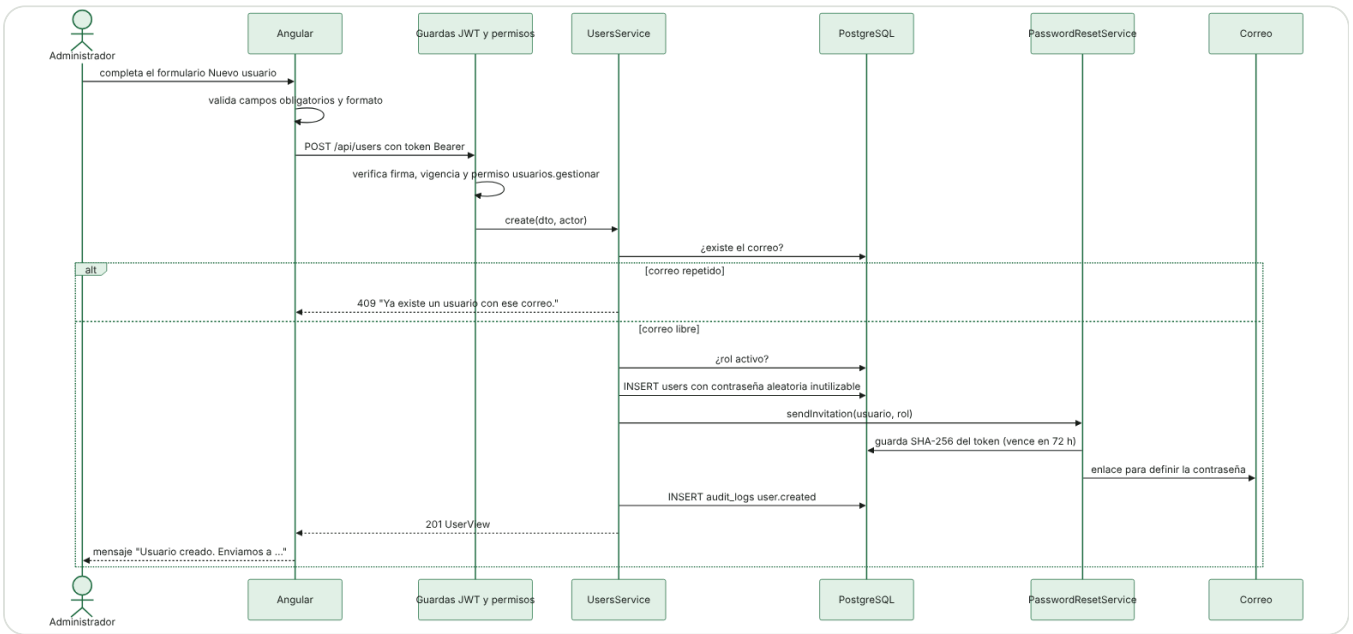
Clases del dominio del sistema completo.



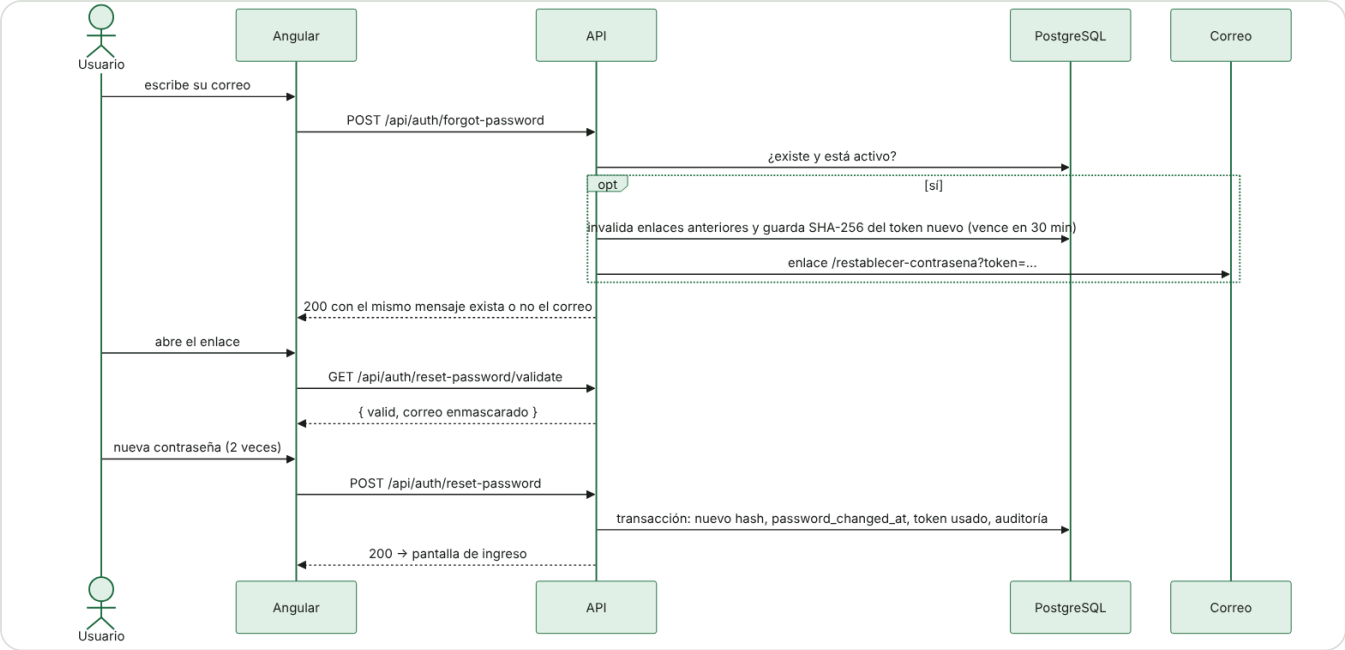
Clases del servicio de aplicación implementado: controladores, servicios, guardas y entidades.



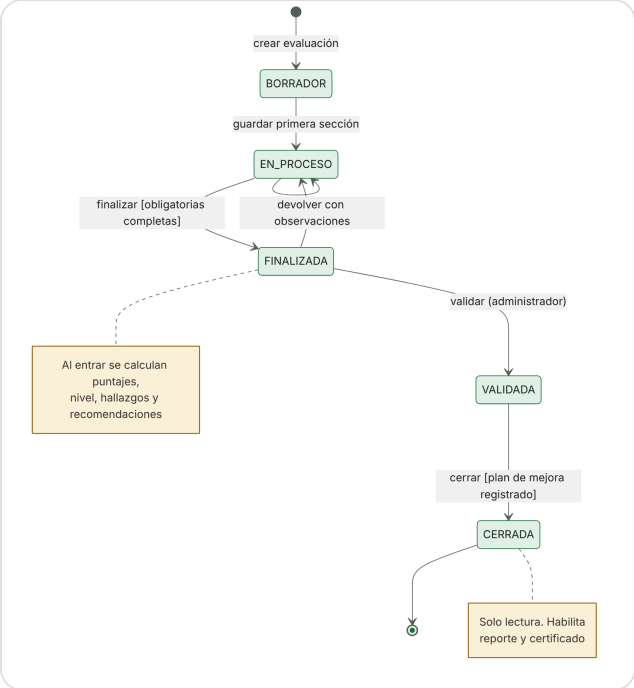
Secuencia de ingreso, con bloqueo tras cinco intentos y respuesta uniforme para no revelar si el correo existe.



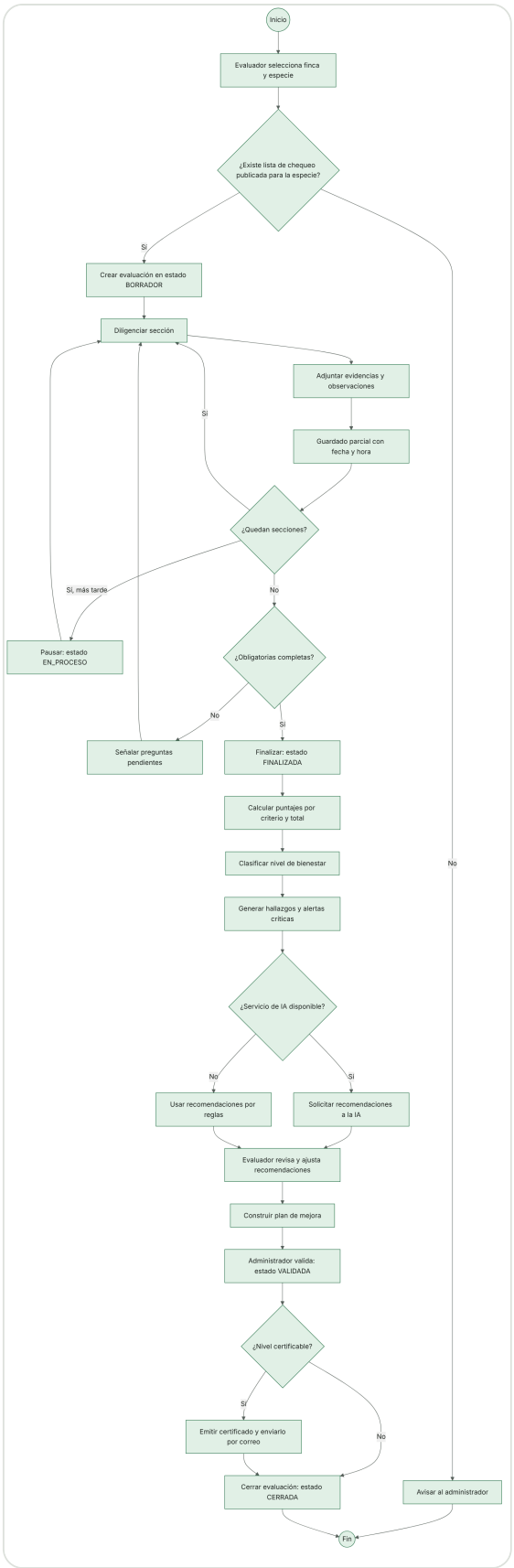
Secuencia de creación de usuario con invitación por correo.



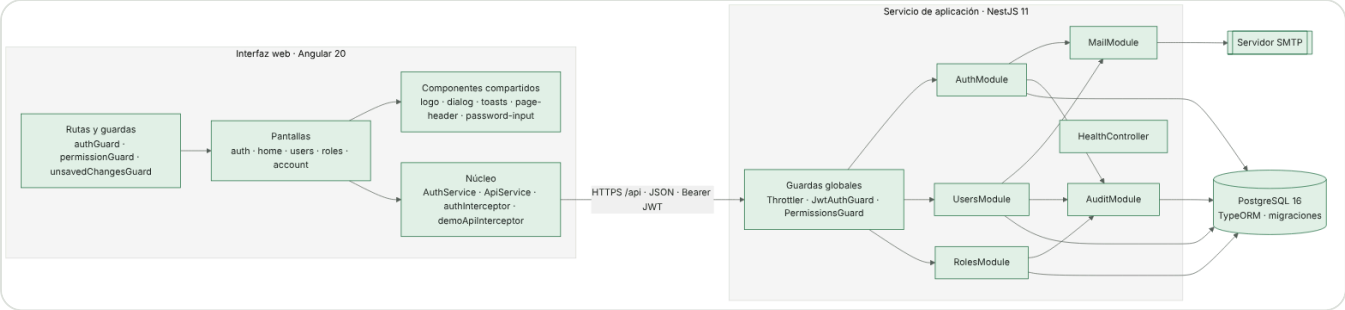
Secuencia de recuperación de contraseña con enlace de un solo uso.



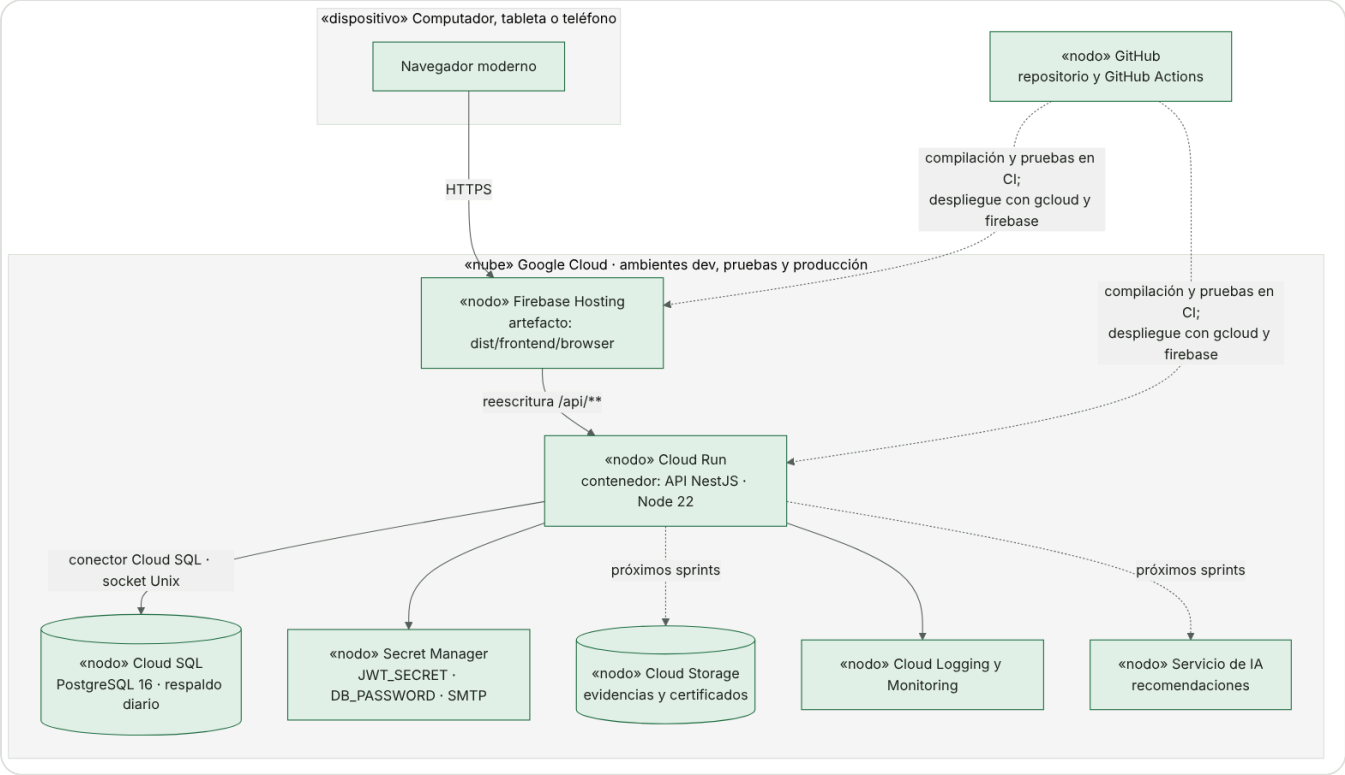
Estados de una evaluación (RF36).



Actividad de una evaluación, con la alternativa por reglas cuando el servicio de IA no está disponible (RNF26).



Componentes de la interfaz y del servicio de aplicación.



Despliegue en Google Cloud. Los nodos con líneas punteadas se incorporan en los próximos sprints.

Diseño de la base de datos

PostgreSQL 16 en Cloud SQL. El modelo cubre los ocho módulos del documento de requerimientos en 29 tablas, en tercera forma normal, y se validó ejecutando el script completo sobre una base vacía.

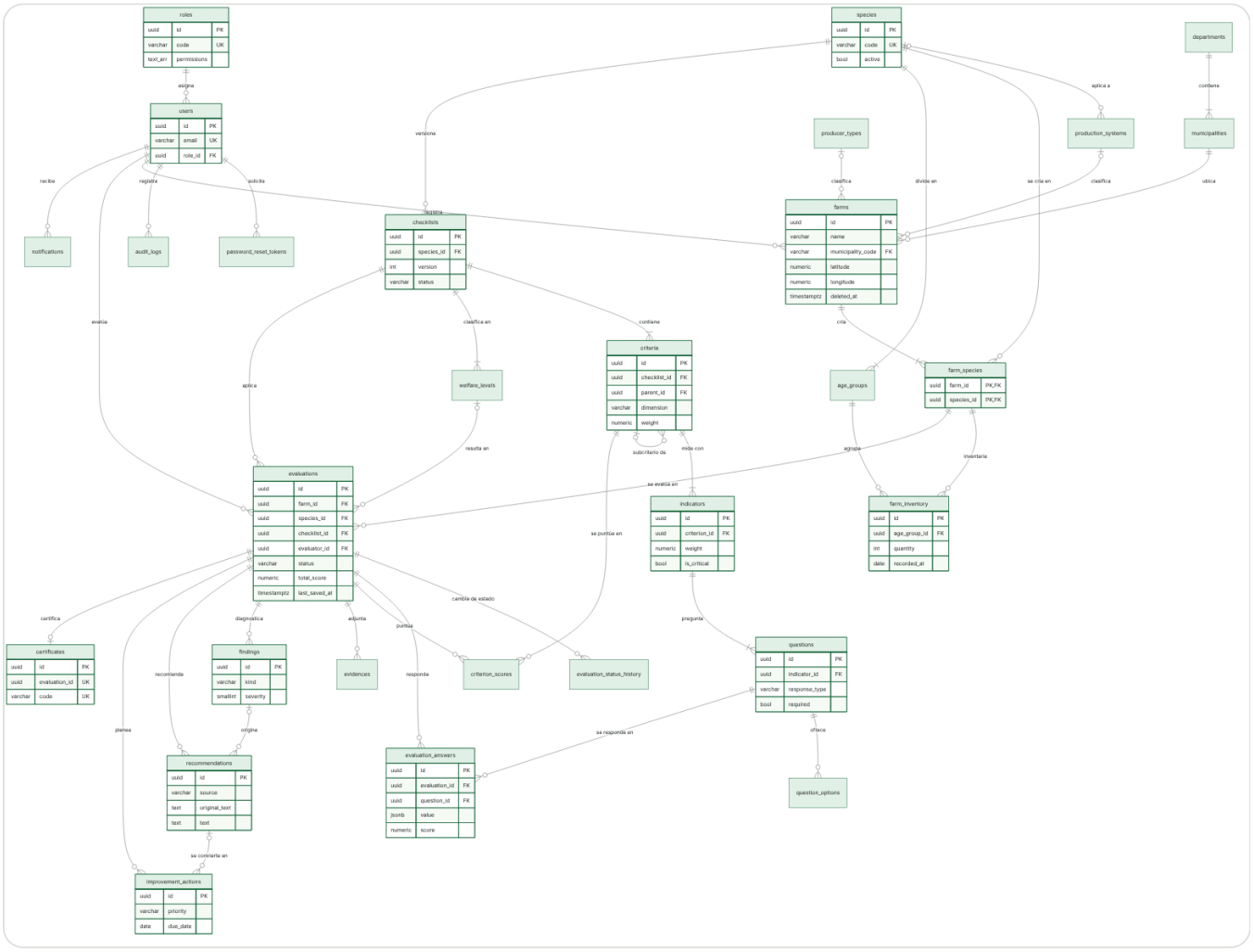
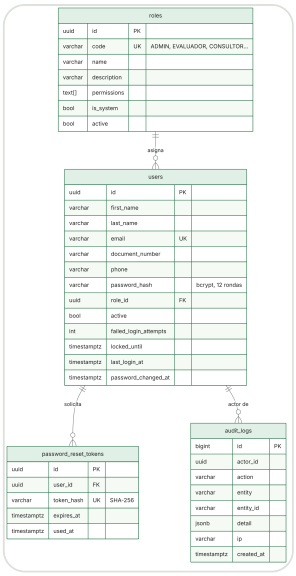


Diagrama entidad relación del sistema completo. El detalle de columnas está en el diccionario de datos.



Módulo de seguridad implementado, con todas sus columnas.

Tablas por módulo

Módulo	Tablas
Seguridad ●	`roles`, `users`, `password_reset_tokens`, `audit_logs`
Catálogos	`departments`, `municipalities`, `species`, `age_groups`, `producer_types`, `production_systems`
Fincas	`farms`, `farm_species`, `farm_inventory`
Parametrización	`checklists`, `criteria`, `indicators`, `questions`, `question_options`, `welfare_levels`
Evaluación	`evaluations`, `evaluation_status_history`, `evaluation_answers`, `criterion_scores`, `evidences`
Diagnóstico	`findings`, `recommendations`, `improvement_actions`
Reportes	`certificates`, `notifications`

Decisiones de diseño

Decisión	Justificación
Listas de chequeo versionadas	Cada evaluación conserva la versión con que se diligenció; un índice único parcial permite una sola versión publicada por especie (RNF28).
Llave compuesta finca y especie	Impide evaluar una especie que la finca no tiene registrada.
Respuestas en JSONB	Un solo campo admite los siete tipos de respuesta del RF12.
Puntajes almacenados	El historial y los tableros no recalculan cada evaluación, y el resultado no cambia si luego se ajustan las ponderaciones.
Texto original de la IA	Se conserva junto al texto revisado por el evaluador, para trazabilidad (RNF25).
Evidencias fuera de la base	Se guarda la ruta del objeto en Cloud Storage (RNF11).
Eliminación lógica de fincas	Conserva el historial de evaluaciones.

Criterios de evaluación

Relación entre los criterios de la entrega y la evidencia que los sustenta.

Criterio	Evidencia
Organización del código	Separación en interfaz y servicio; módulos por funcionalidad; guardas globales; configuración validada; migraciones; componentes reutilizables; 70 pruebas automatizadas; manuales de instalación y de usuario.
Uso adecuado de Git	Historial en `main` con commits descriptivos por cambio completo; integración continua en GitHub Actions; `.gitignore` y `.env.example` sin secretos; etiqueta `v0.2.0`; estrategia de ramas documentada.
Calidad del diseño	Sistema visual propio documentado; capturas en dos tamaños y dos modos; accesibilidad verificada; diez diagramas UML; modelo de datos completo en tercera forma normal con script validado y diccionario de datos.

Revisión con la lista de buenas prácticas

Práctica	Cómo se atendió
Requerimientos y módulos definidos	Los requerimientos funcionales y no funcionales del documento se agruparon en ocho módulos; el plan de trabajo indica el estado de cada uno.
Arquitectura	Documentada en `docs/ARQUITECTURA.md` y en los diagramas de componentes y despliegue.
Revisión del código y pruebas	Pruebas unitarias, de integración y de extremo a extremo en verde antes de cada envío.
Lectura de registros	Errores del servicio en Cloud Logging; bitácora de auditoría propia para acciones críticas.
Seguridad	bcrypt, JWT con revocación al cambiar contraseña o rol, bloqueo por intentos, límite de peticiones y secretos fuera del código.
Diseño de la base de datos	Modelo completo antes de construir los módulos siguientes.
Validación de datos	En la interfaz, en los DTO del servicio y con restricciones en la base.
Diseño móvil	Probado a 390 px de ancho en cada pantalla; formularios como hojas inferiores.
Funciones esenciales primero	El primer sprint entrega solo la seguridad, base de todos los módulos.
Costos y ambiente de producción	Servicios administrados que escalan a cero (Cloud Run) y una instancia pequeña de Cloud SQL; ambientes separados de desarrollo, pruebas y producción.
Casos reales y revisión final	Datos de ejemplo realistas, modo demostración y revisión de diseño con las guías de Apple.